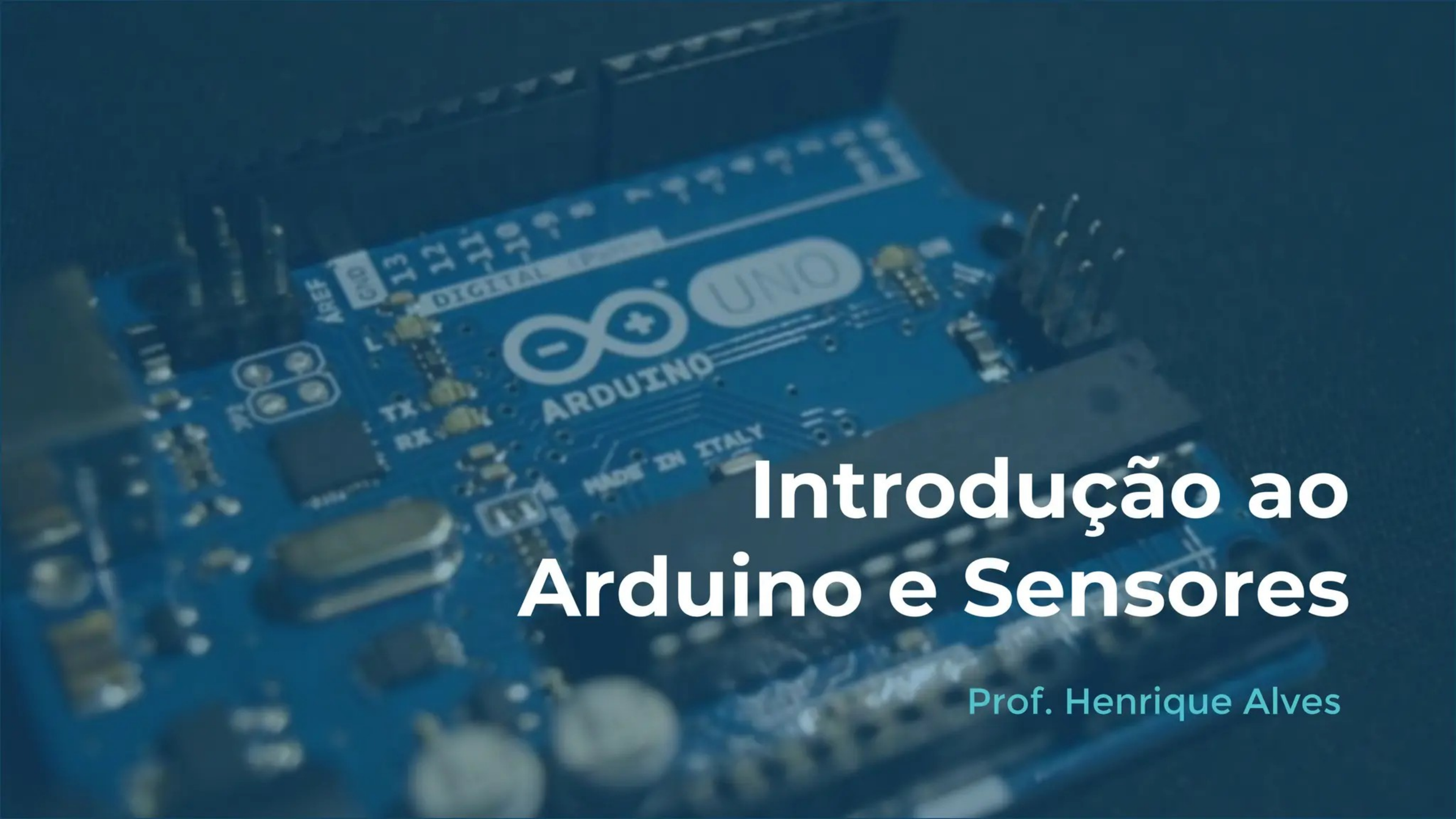



# Introdução ao Arduino e Sensores

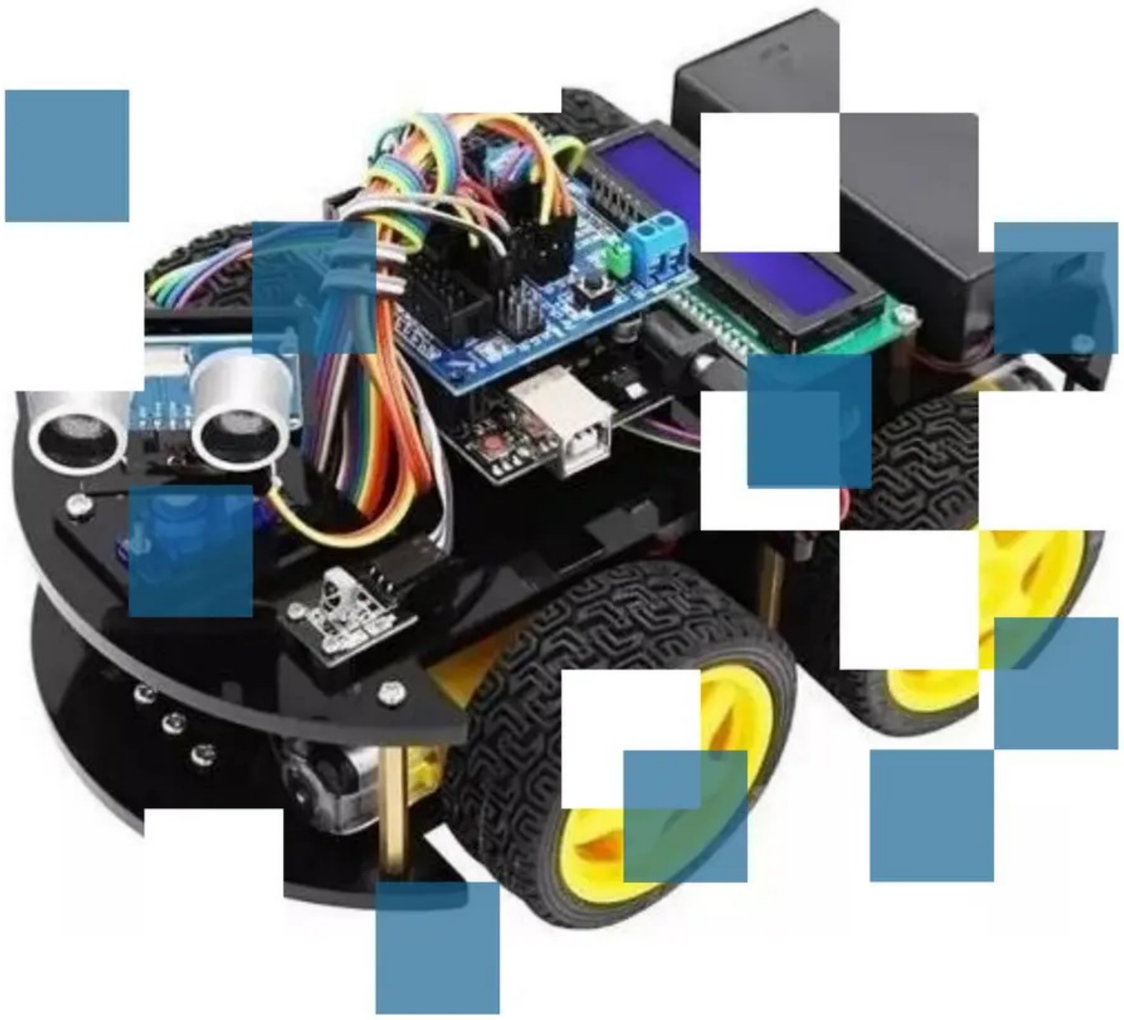
Prof. Henrique Alves




# O que é o Arduino?

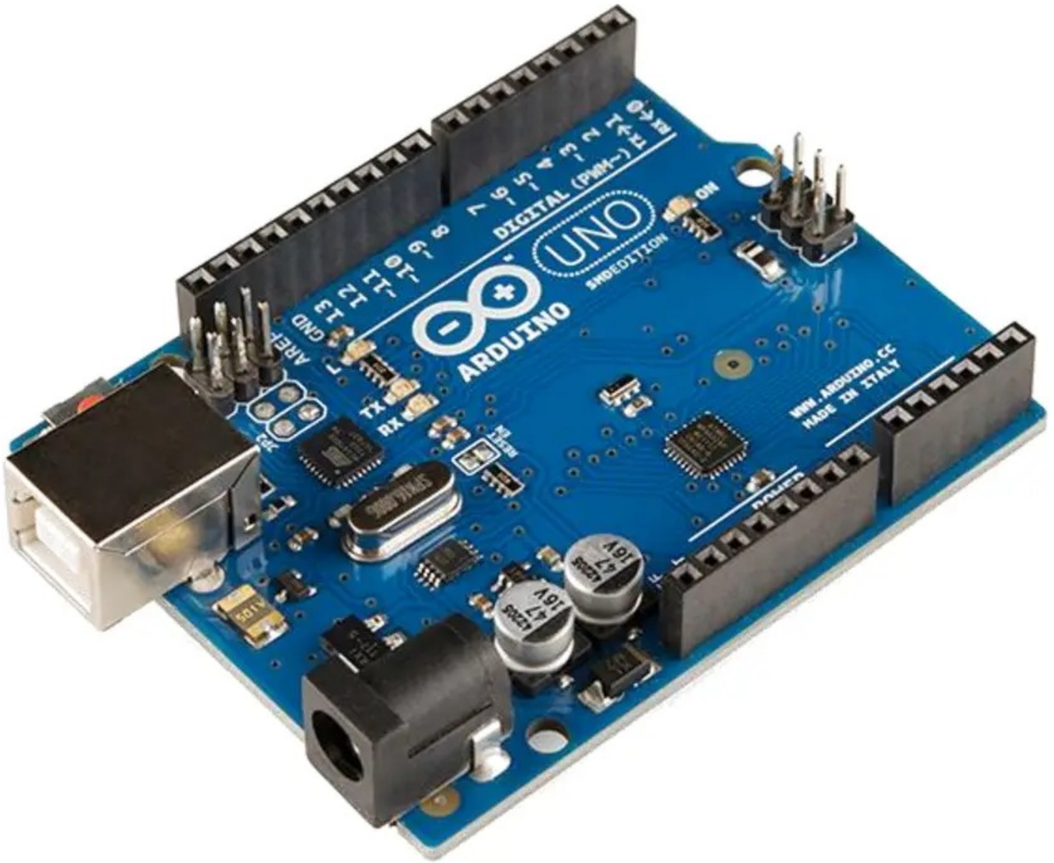
Arduino é uma **plataforma eletrônica** de código aberto baseada em hardware e software fáceis de usar. As placas Arduino são capazes de ler entradas *( luz em um sensor, um dedo em um botão ou uma mensagem do Twitter),* e transformá-la em uma saída - ativando um motor, ligando um LED, publicando algo online.

Saiba mais...



# O que é o Arduino?

O Arduino foi criado em **2005** e seu objetivo era elaborar um dispositivo que fosse ao mesmo tempo barato, funcional e fácil de programar, sendo dessa forma acessível a estudantes e projetistas amadores. Além disso, foi adotado o conceito de **hardware livre**, o que significa que qualquer um pode montar, modificar, melhorar e personalizar, partindo do mesmo hardware básico.




# O que é a Placa?

A plataforma Arduino criou uma **placa** composta por um **microcontrolador Atmel** e circuitos de **entrada/saída,** e que pode ser facilmente conectada à um computador e programada via **IDE** *(Integrated Development Environment, ou Ambiente de Desenvolvimento Integrado)* utilizando uma linguagem baseada em **C/C++,** sem a necessidade de equipamentos extras além de um cabo USB.

# Tipos de Placas

Cada placa depende do projeto a ser desenvolvido.



























# Tipos de Placas



ARDUINO YÚN



ARDUINO ETHERNET


ARDUINO INDUSTRIAL 101



MKR FOX 1200



MKR WAN 1300



MKR GSM 1400



MKR WiFi 1010


UNO Wifi REV2

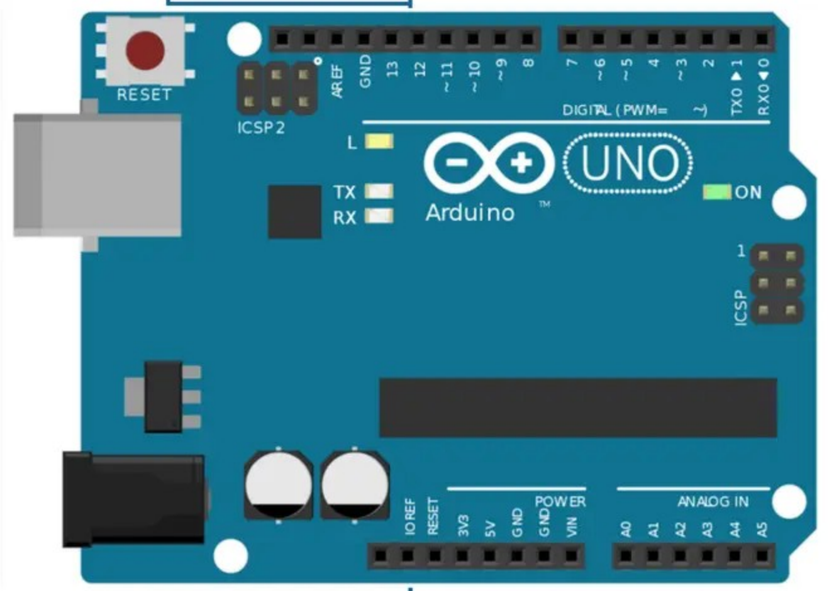


MKR NB 1500

# Porque utilizar o Arduino?

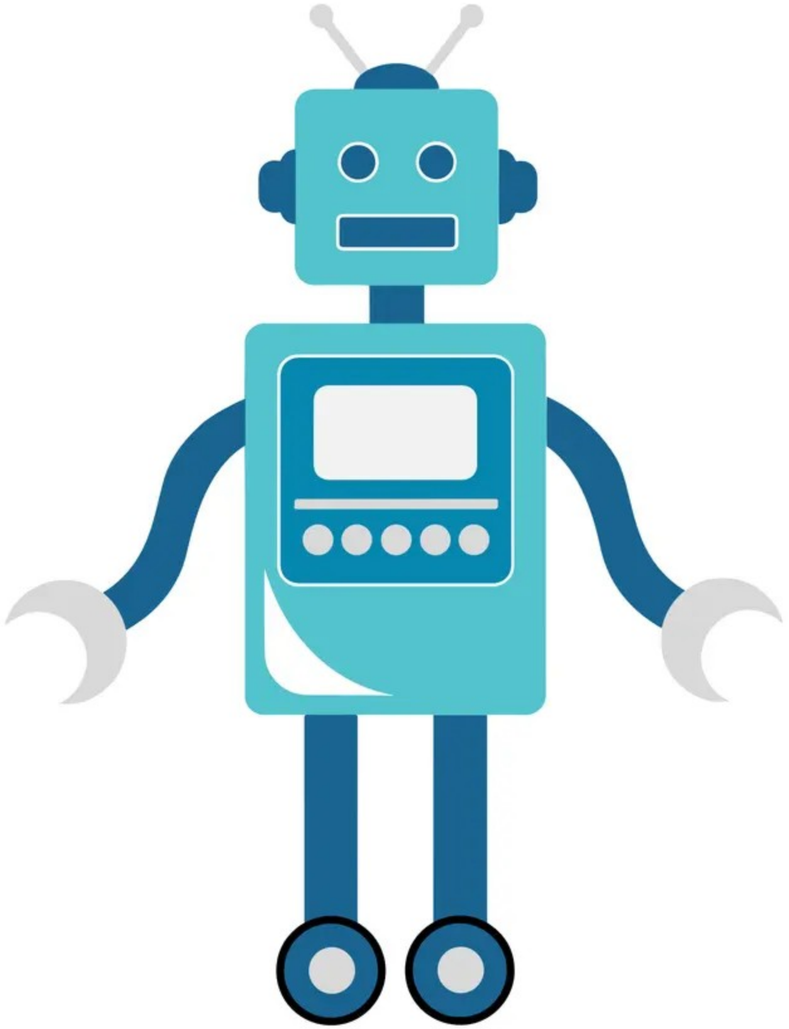
Fácil de programar

Baixo custo de prototipagem




Softwares de simulação gratuitos disponíveis

Extensa comunidade de desenvolvedores



# O que pode ser feito com Arduino?

- A lista de possibilidades é praticamente infinita. Você pode automatizar sua casa, seu carro, seu escritório, criar um novo brinquedo, um novo equipamento ou melhorar um já existente. Tudo vai depender da sua **criatividade.**

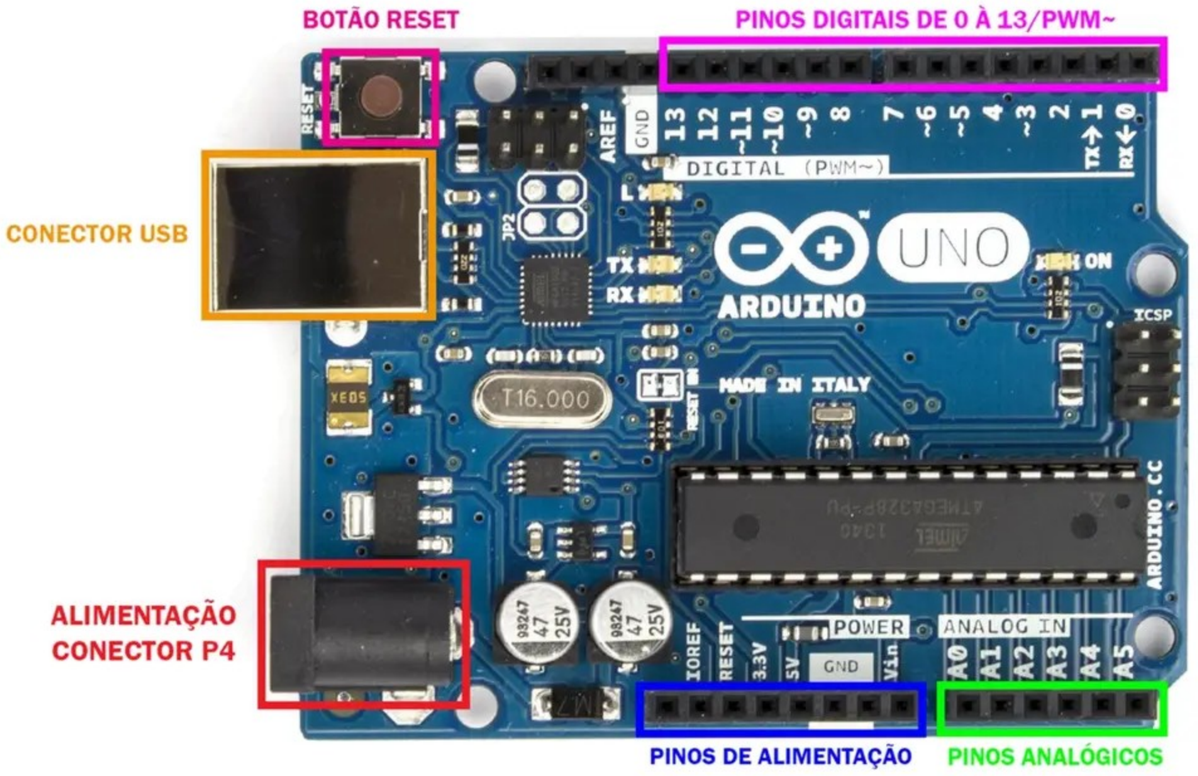
- Para isso, o Arduino possui uma quantidade enorme de sensores e componentes que você pode utilizar nos seus projetos. Grande parte do material utilizado está disponível em módulos, que são pequenas placas que contém os sensores e outros componentes auxiliares como resistores, capacitores e leds. **Saiba mais.**

# Entendendo a placa Arduino

- **GND:** abreviação de *"ground"* (terra em inglês). Existem vários pinos de GND no Arduino, qualquer um deles pode ser utilizado para aterrar seu circuito;
- **5V e 3V3:** o pino de 5V fornece 5 volts de energia e o pino de 3V3 fornece 3,3 volts.

- **PINOS ANALÓGICOS:** a área dos pinos rotulada com "ANALOG IN" (A0 a A5 no UNO) é para os pinos analógicos. Esses pinos podem ler o sinal de um sensor analógico (como um sensor de luminosidade) e convertê-lo em um valor digital para usar no código;

- **PINOS DIGITAIS:** a área dos pinos rotulada com "DIGITAL" (0 a 13 no UNO) é para os pinos digitais. Esses pinos podem ser usados tanto para entrada digital (como dizer se um botão está pressionado) como para saída digital (como alimentar um LED);
- **PWM:** você pode ter notado o til (~) ao lado de alguns dos pinos digitais (3, 5, 6, 9, 10 e 11 no UNO, esses são pinos de Modulação por Largura de Pulso (PWM).




Os pinos na sua placa Arduino são os lugares onde você conecta os fios para construir um circuito. O Arduino tem vários tipos diferentes de pinos, cada um dos quais é rotulado na placa e usado para diferentes funções.

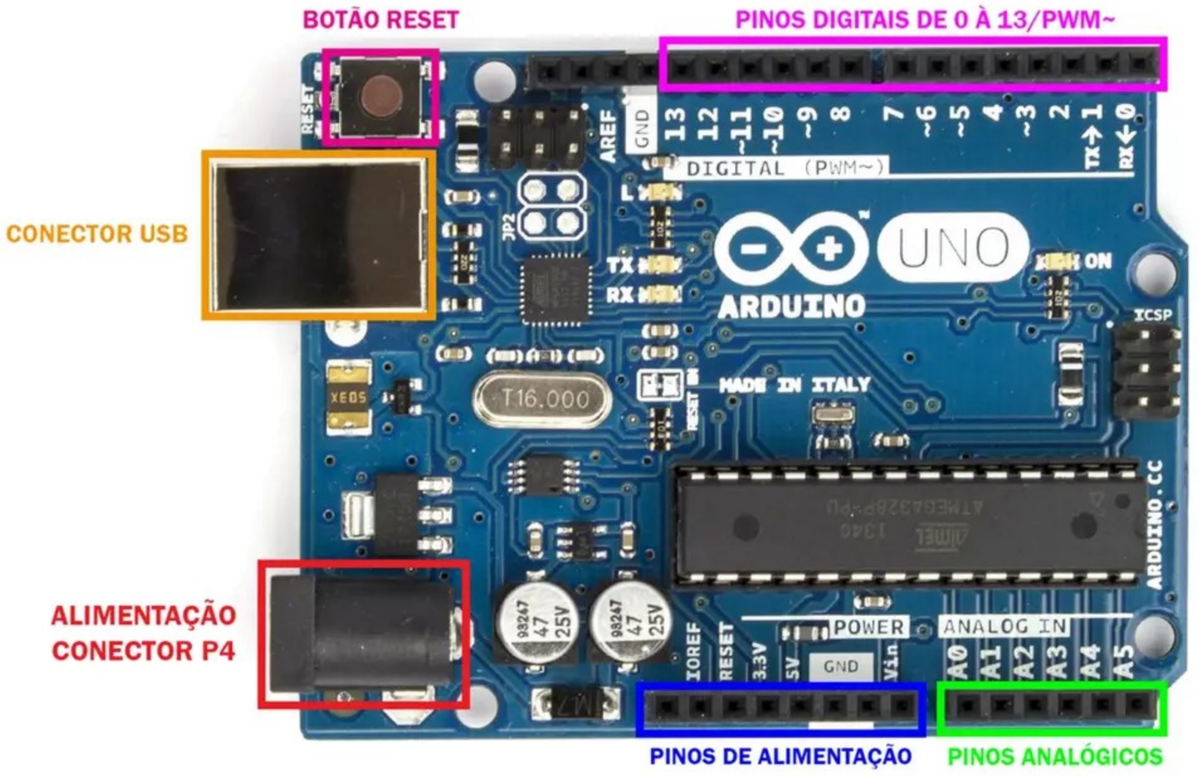
# Entendendo a placa Arduino

- **BOTÃO RESET:** Conecta temporariamente o pino de reset (RST) ao GND e reiniciará o programa carregado no Arduino. Isso pode ser muito útil se o seu código não repetir, mas você quer testá-lo várias vezes. **NOTA:** *o reset não apaga o código carregado na placa, ele serve somente para reiniciar a execução do programa.*

- **CONEXÃO USB B:** Além de fazer a alimentação também é como você irá carregar o código em sua placa Arduino.

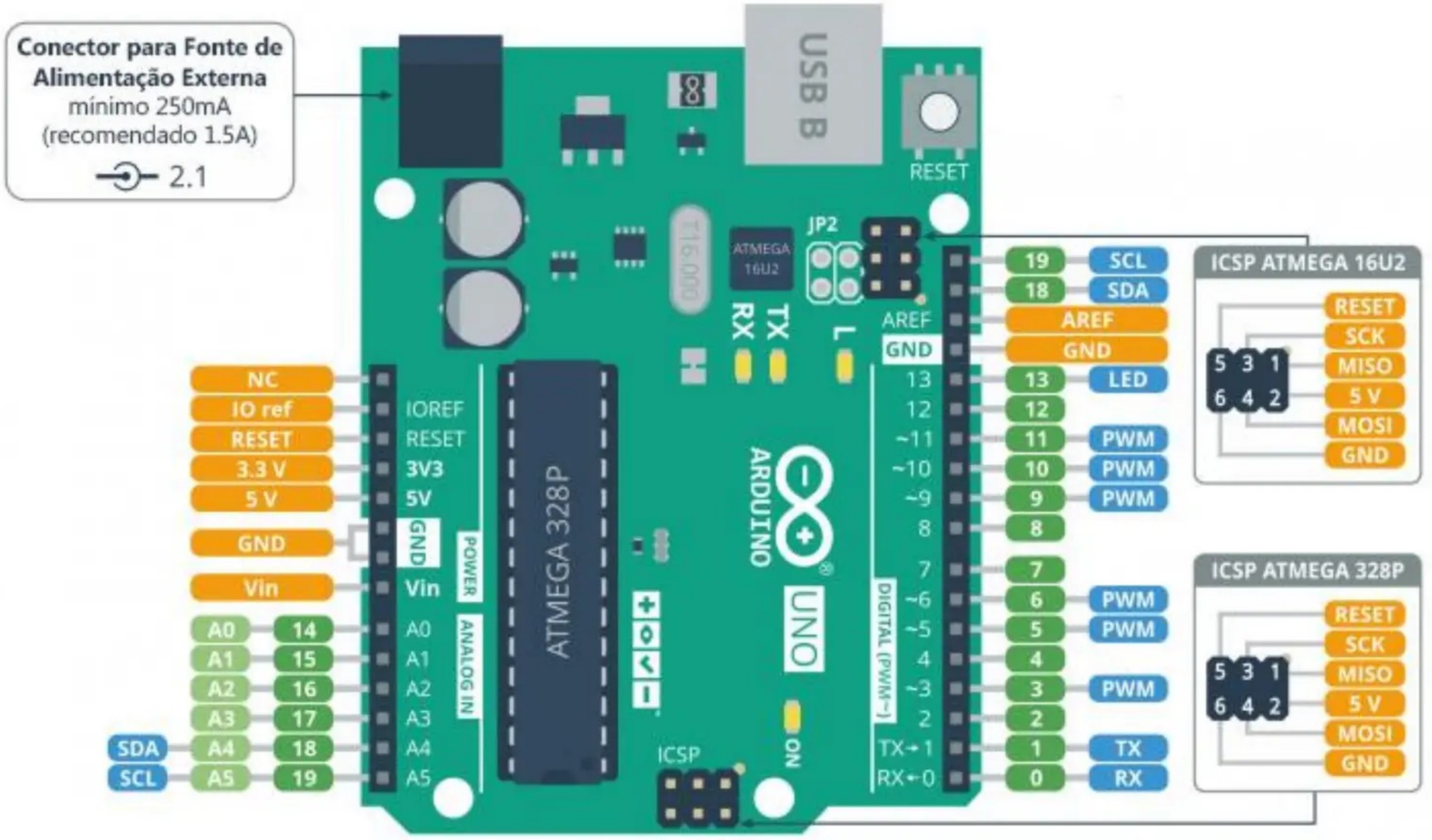
- **CONECTOR P4:** Interface para fonte de alimentação externa de 9v à 15v.

- **NOTA**: **NÃO** use uma fonte de alimentação **superior a 15 V**, pois você irá superar a tensão máxima do seu Arduino ***(e assim queimá-lo)***. A tensão recomendada para a maioria dos modelos Arduino é de **6 V a 12 V.**




Os pinos na sua placa Arduino são os lugares onde você conecta os fios para construir um circuito. O Arduino tem vários tipos diferentes de pinos, cada um dos quais é rotulado na placa e usado para diferentes funções.

# Entendendo a placa Arduino





# Softwares e Drives

## IDE Arduino e Simulador Tinkercad

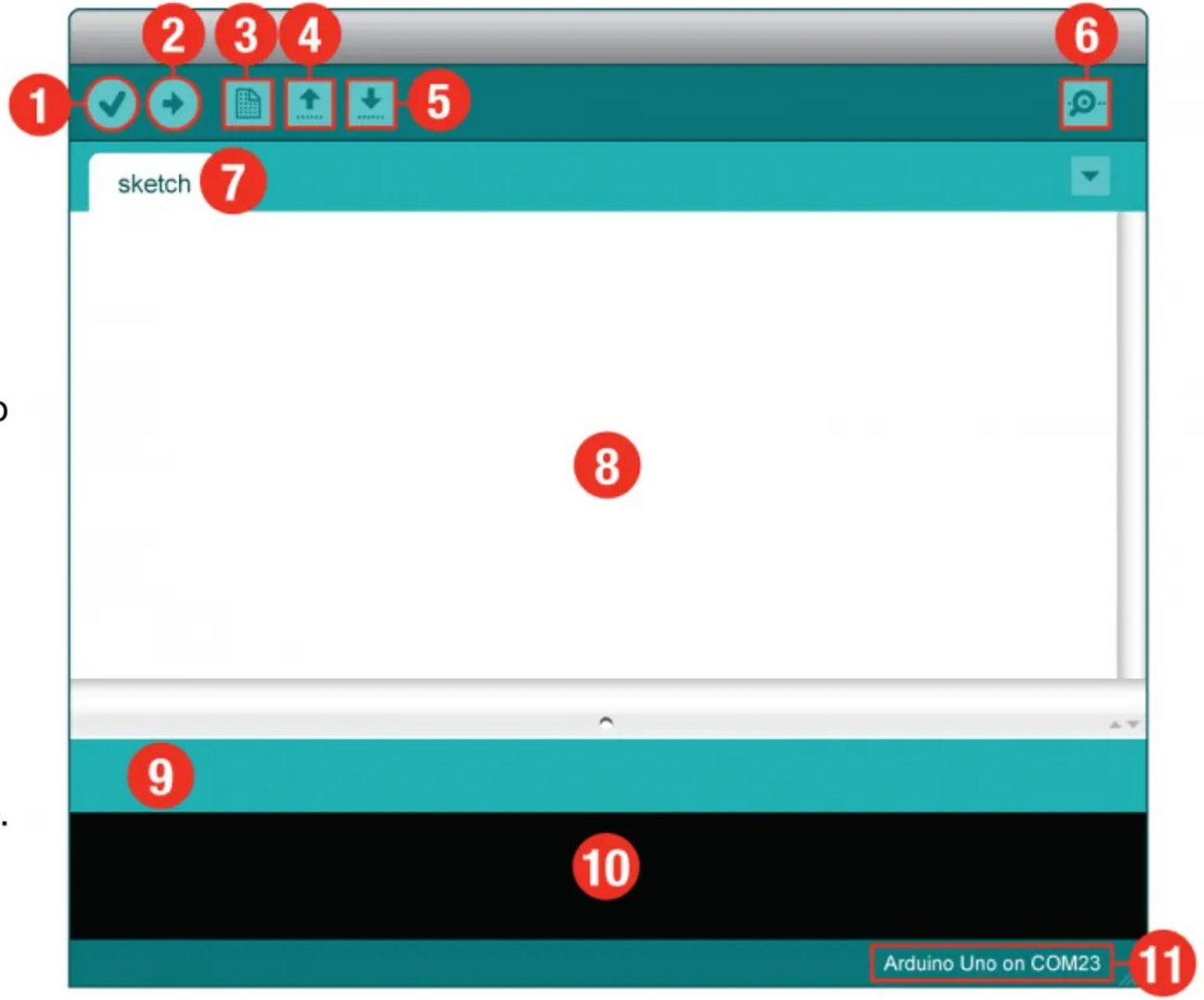
# Instalando o Arduino IDE

- O primeiro passo para começar a trabalhar com o Arduino é instalar a versão mais atual do ambiente de desenvolvimento (IDE) do Arduino, onde todos os código serão desenvolvidos. O ambiente de desenvolvimento pode ser baixado gratuitamente de www.arduino.cc.

- A Arduino IDE é um software de código aberto *(open source),* ou seja, todo o seu código fonte está disponível para utilização e pode ser adaptado para diferentes fins. Todas as vezes que você faz um download na página oficial do software, uma tela aparece solicitando uma doação para ajudar a acelerar o desenvolvimento do projeto. Não é obrigatório doar para baixar a IDE, basta clicar em *JUST DOWNLOAD* para fazer o download sem custo algum.

# Introdução à Arduino IDE

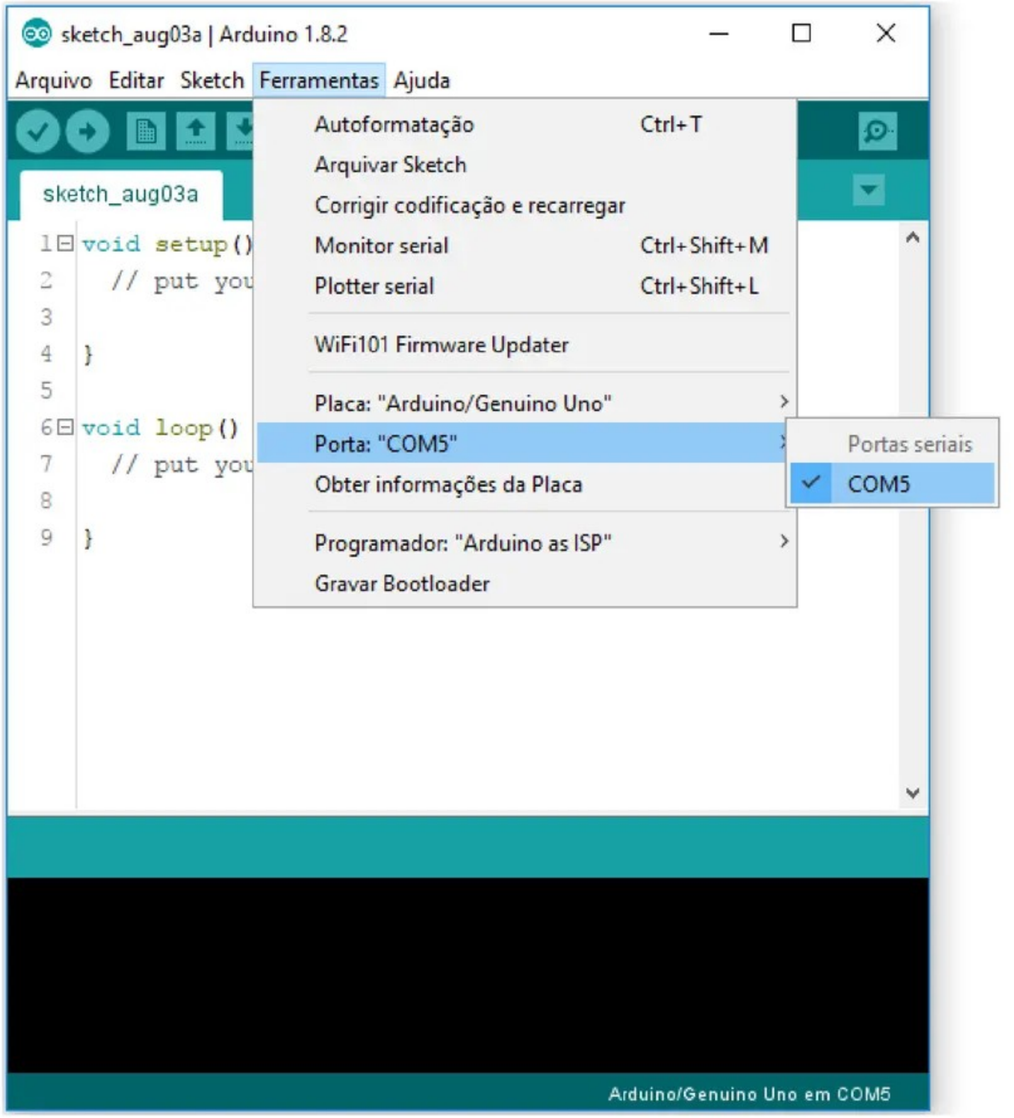
1. **Verificar:** compila e aprova o seu código. O compilador detectará erros de sintaxe (como ponto e vírgula ou parênteses faltantes);
2. **Carregar:** envia o seu código para a placa Arduino;
3. **Novo:** abre uma nova janela de código;
4. **Abrir:** permite abrir um esboço existente;
5. **Salvar:** salva o esboço atualmente ativo;
6. **Monitor serial:** abre uma janela que exibirá qualquer informação serial que sua placa Arduino está transmitindo para o seu computador
7. **Nome do esboço:** mostra o nome do esboço no qual você está trabalhando atualmente;
8. **Área do código:** esta é a área onde você compõe o código para o seu esboço;
9. **Área de mensagens:** é onde a IDE diz se houve algum erro no seu código ou se o código foi compilado e carregado corretamente;
10. **Console de texto:** mostra mensagens de erro completas. O console de texto é muito útil para a depuração;
11. **Placa e porta serial:** mostra a placa e a porta serial selecionadas.



# Instalando o Drive da Placa

1. Após a instalação da IDE em seu computador, conecte o cabo USB presente no kit em sua placa e depois a uma porta USB do computador. A placa deve ser reconhecida automaticamente pelo computador, uma vez que a IDE já foi instalada.

2. Quando o computador reconhece a placa Arduino, uma porta de comunicação é criada para que a IDE consiga se comunicar com a placa. No Windows você pode verificar a porta criada acessando o Gerenciador de Dispositivos do sistema. Esta porta terá o nome **COMx**, onde **x** é um número, e esta porta deverá ser selecionada na IDE do Arduino, no menu Ferramentas > Porta.

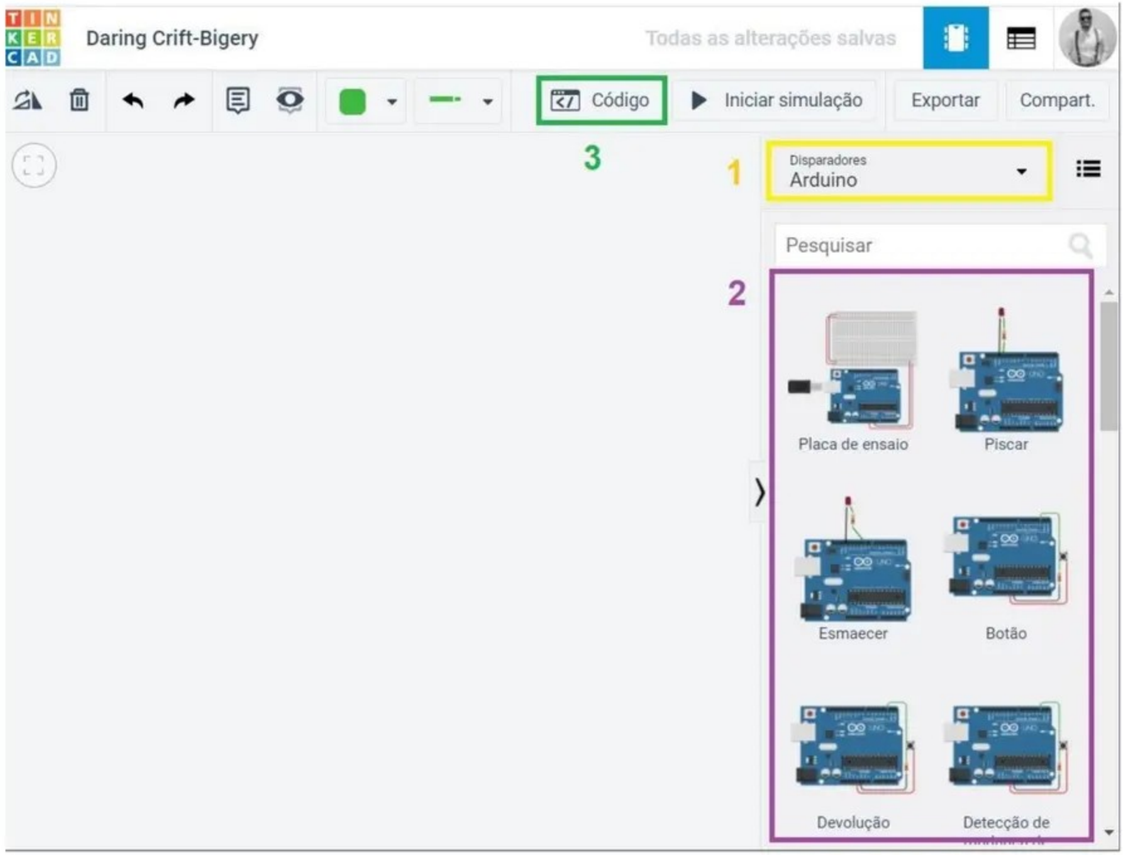


# Simulando Arduino com Tinkercad

1. É possível e simular vários projeto com Arduino com o Tinkercad. No menu de componentes temos projetos prontos em disparadores>Arduino

2. O Tinkercad possui vários exemplos de projetos com Arduino, mas podemos fazer nos projetos do zero também.

3. É possível trabalhar com programação de placas no menu código.

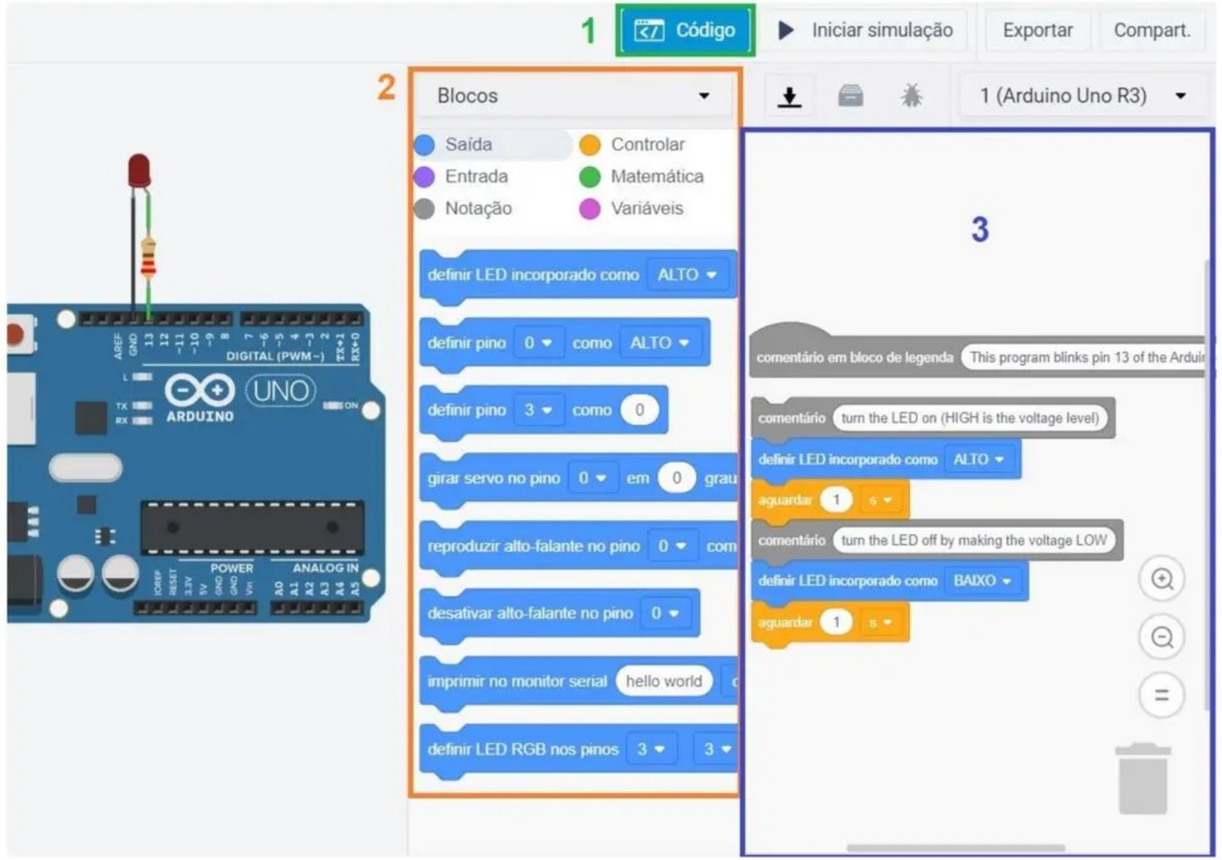


# Simulando Arduino com Tinkercad

1. Temos disponível um editor de códigos para Arduino, onde podemos programar usando blocos ou texto;

2. Opções de comando via blocos de programação com saídas de dados, entrada, funções matemática e etc.;

3. Área destinada a edição de códigos tanto com blocos como com texto.






# Programação com Arduino

## Lógica de programação com C/C++

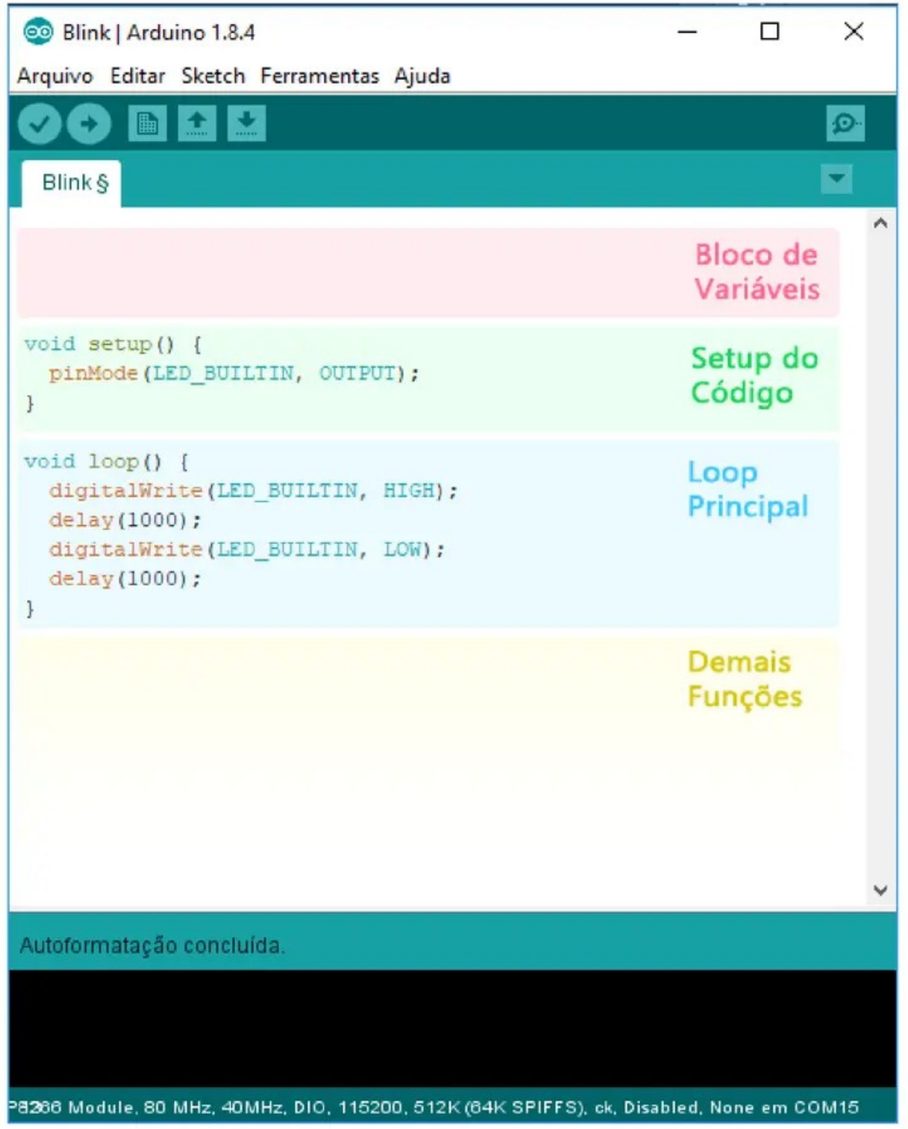
# Programação com Arduino

- A linguagem de programação do Arduino pode ser dividida em três partes principais: estruturas, valores (variáveis e constantes) e funções.
- **Lógica de programação** é basicamente o fato de criar uma sequência que deve ser seguida ordenadamente a fim de chegar a um objetivo. Não é de hoje que você segue sequências predeterminadas, você tem feito isso sua vida inteira. Quer dizer que você é um programador nato? Quase isso! Quer um exemplo? Responda rápido:
  1. Quando você quer enviar uma mensagem para alguém e seu celular está desligado, primeiro você liga o celular ou primeiro você digita a mensagem?
  2. Quando você quer comer uma banana, primeiro você descasca a banana ou primeiro você morde ela?
  3. Quando você vai preparar um macarrão instantâneo, você coloca o macarrão na água fria ou na água fervendo?

# Programação com Arduino

- **A estrutura básica *(sketch)* onde escrevemos nosso código consiste em:**

1. **Bloco de Variáveis:** onde ficarão as variáveis globais que utilizaremos no código e chamadas de bibliotecas;
2. **Setup do Código:** onde informaremos se os pinos serão entradas, saídas, abriremos a comunicação serial, etc;
3. **Loop Principal:** a parte do código que ficará rodando em um laço (loop) infinito;
4. **Demais Funções:** caso tenhamos outras funções a serem usadas no código.



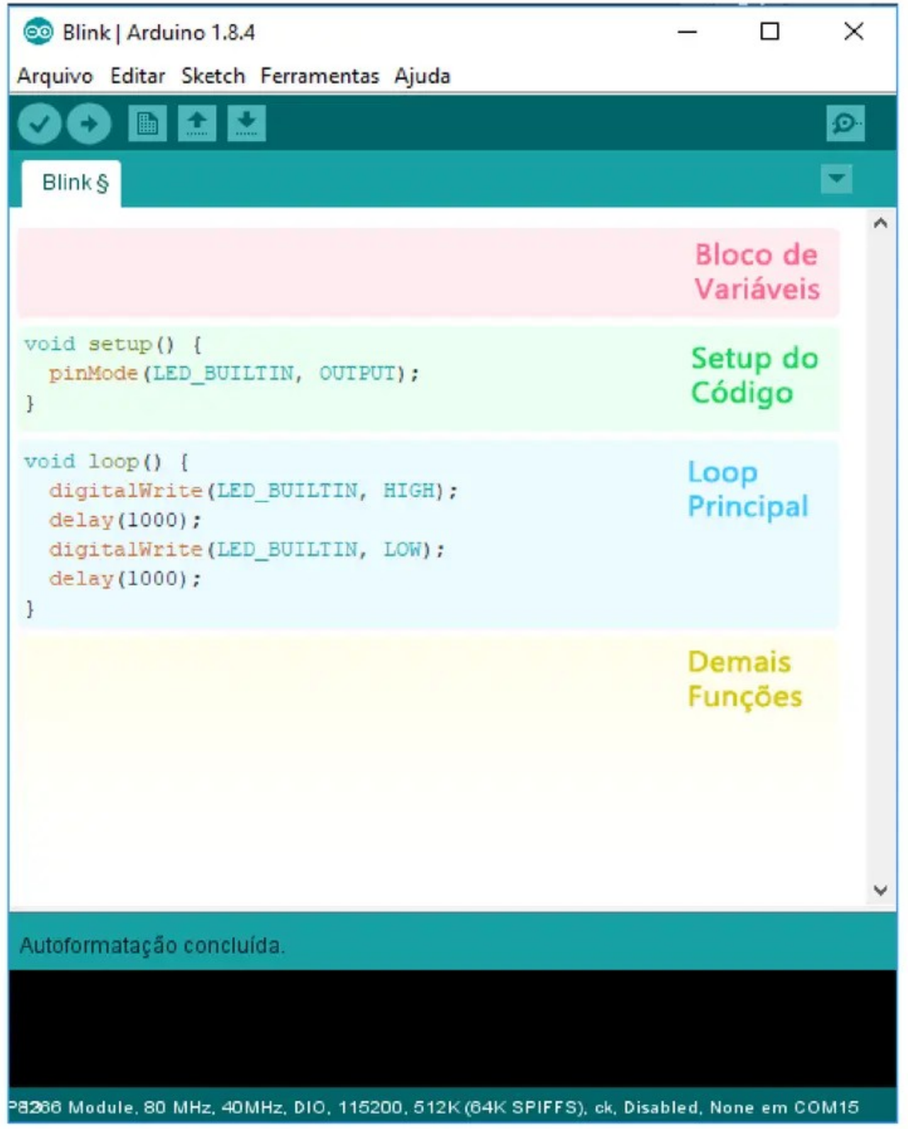
# Programação com Arduino

- Note que **setup()** e **loop()** também são funções, e são as únicas funções obrigatórias em **todos** os códigos em Arduino.

- As palavras **setup** e **loop** no código têm símbolos de **parêntesis** abrindo e fechando, além de chaves. Veja que nas funções **pinMode** também existem parêntesis, porém com informações dentro. Estes parêntesis ao lado de palavras servem para o **compilador** entender se aquela função tem algum **parâmetro**, ou seja, alguma informação adicional que interfere no funcionamento da função.

- **EsTa** palavra é igual a **eSta**?

  ✓ Quando vamos programar na plataforma Arduino precisamos tomar cuidado com a forma que escrevemos as palavras já que o compilador do *código é case sensitive*, ou seja, sensível a letras maiúsculas e minúsculas.



# Programação com Arduino

- **Declaração de Variáveis**

  Declaração de variável é um termo elegante que significa que você precisa digitar o nome de cada entrada ou saída que deseja usar em seu sketch.

  ```
  int nome_variavel = valor;
  ```

- **A função setup**

  Essa função é executada uma vez, sempre que o Arduino for ligado. Esse geralmente é o lugar em que podemos determinar quais **das variáveis declaradas** são entradas ou saídas por meio do comando **pinMode(**pin, modo**).**

  ```
  void setup() {
    pinMode(my_led, OUTPUT);
  }
  ```

# Programação com Arduino

- **A função loop**

  Essa função é o lugar em que o código principal será colocado e **executado repetidas vezes** e continuamente, até que o Arduino seja desligado. É aí que informamos o que o Arduino deve fazer no processo. Toda vez que o processo atinge o fim dessa função de repetição, volta ao início.

  Nesse exemplo, a função **loop** simplesmente **pisca o LED** de forma intermitente usando a função **delay(ms)** (atraso). A alteração do primeiro delay(1000) determina por quanto tempo o LED deve permanecer ligado, enquanto a alteração do segundo delay(1000) determina por quanto tempo o LED deve permanecer desligado.

```
void loop() {
// início do loop, repita o seguinte:
digitalWrite(my_led, HIGH); // mude LED para ligado
delay(1000); // espere por 1 s
digitalWrite(my_led, LOW); // desligue o LED
delay(1000); // espere por 1 s
// termine o loop e volte para o início
}
```

# Programação com Arduino

- Algumas funções da linguagem

**Entradas e Saídas Digitais**

- digitalRead()
- digitalWrite()
- pinMode()

**Entradas e Saídas Analógicas**

- analogRead()
- analogReference()
- analogWrite()

**Funções Temporizadoras**

- delay()
- delayMicroseconds()
- micros()

**Funções Matemáticas**

- abs()
- max()
- min()

Saiba mais...

# Programação com Arduino

- Algumas estruturas da linguagem

**Estruturas de Controle**

- break
- do...while
- if
- else
- for

**Outros Elementos da Sintaxe**

- /* */ (comentário em bloco)
- // (comentário)
- ; (ponto e vírgula)
- {} (chaves)

**Operadores Aritméticos**

- * (multiplicação)
- + (adição)
- - (subtração)
- / (divisão)
- = (operador de atribuição)

**Operadores de Comparação**

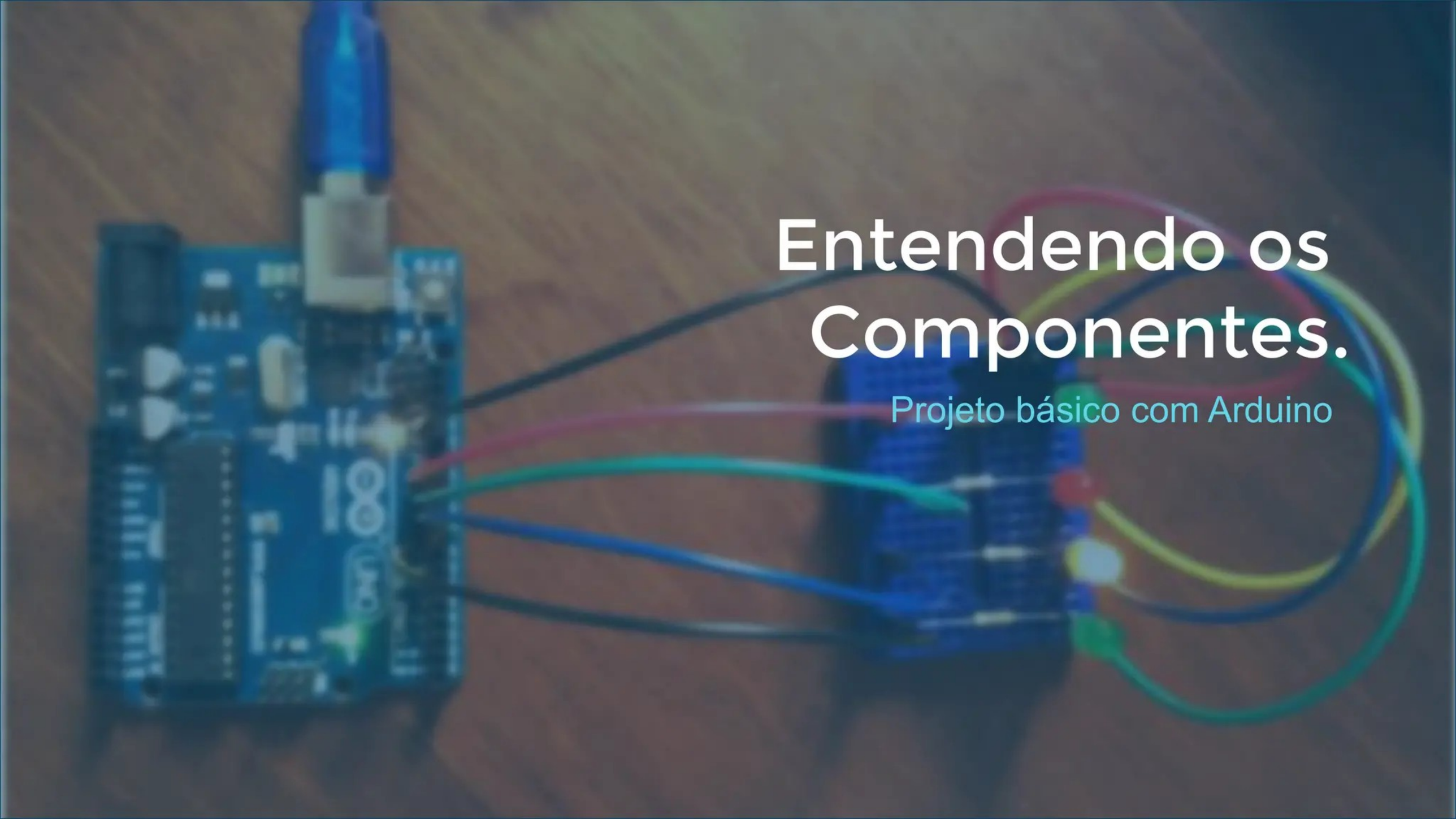
- != (diferente de)
- < (menor que)
- <= (menor que ou igual a)

Saiba mais...



# Projeto 01 – Pisca LED

Projeto básico com Arduino



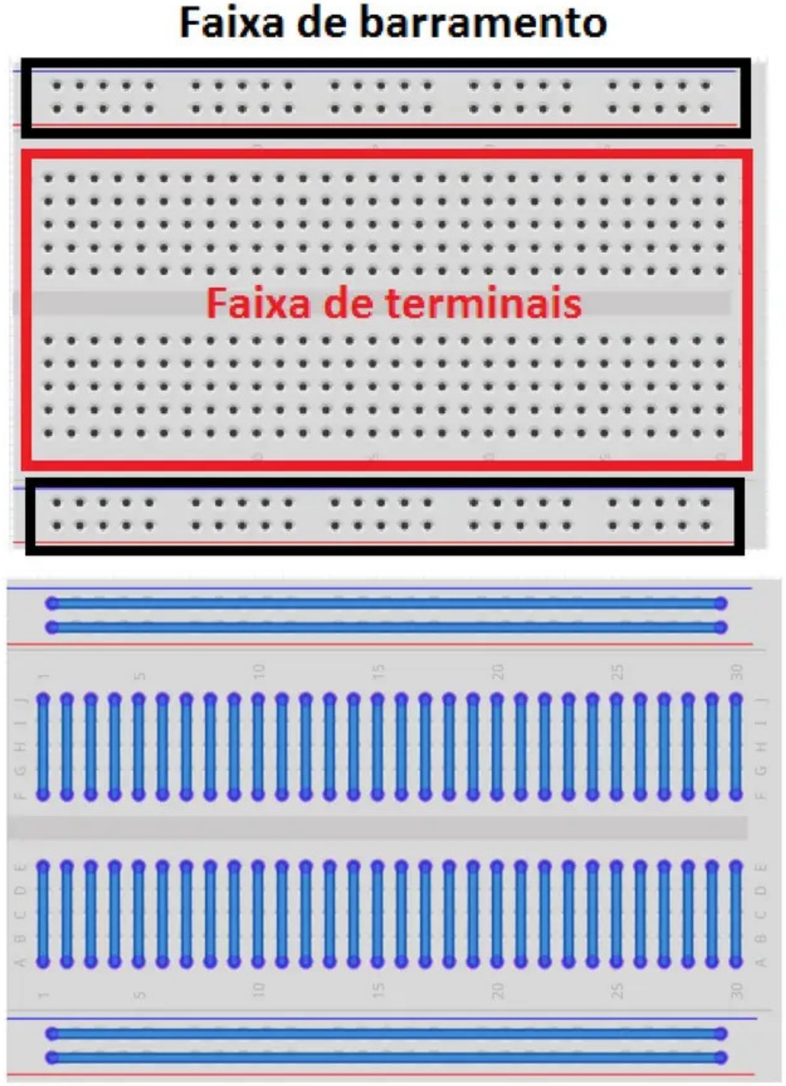
# Entendendo os Componentes.

Projeto básico com Arduino

# Componentes - Protoboad

- Pensando em facilitar e agilizar testes, foi criada a "placa de ensaio", mais conhecida como **protoboard**, que basicamente é uma placa repleta de **contatos** que facilitam a montagem e testes de circuitos eletrônicos.

- A faixa de **barramento** é os pinos destinados a alimentação do circuito, ou seja, recebem os pinos de **VCC** e **GND** e tem seus furos **interligados horizontalmente**. Já a faixa de terminais é a área destinada aos componentes e tem seus furos **ligados verticalmente.**

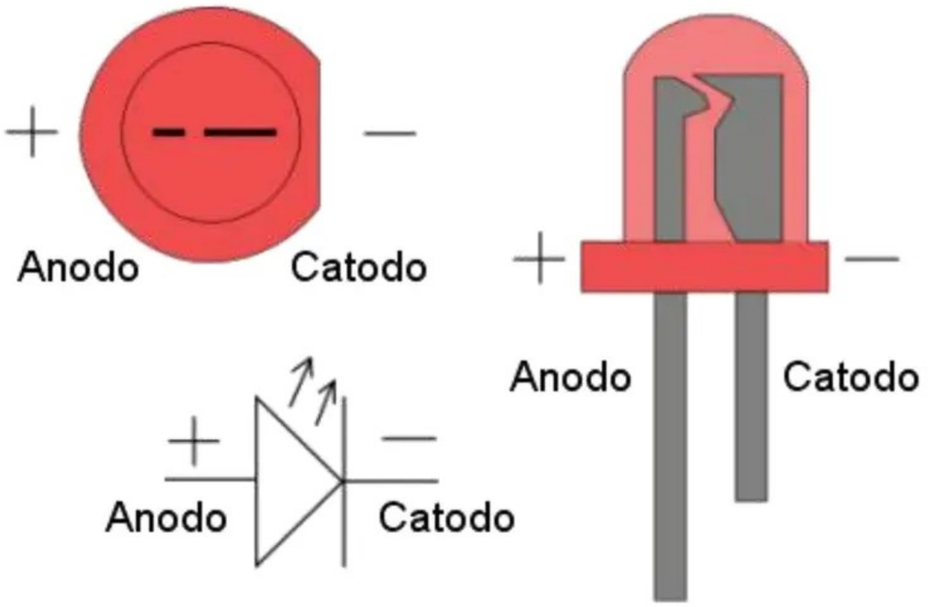


# Componentes - LED

- O **diodo** emissor de luz (sigla **LED**, em inglês*: light-emitting diode*), é usado para a emissão de luz em locais e instrumentos onde se torna mais conveniente a sua utilização no lugar de uma lâmpada.

- Trata-se de um componente eletrônico capaz de emitir luz visível transformando energia elétrica em energia luminosa. Esse processo é chamado de **eletroluminescência**

- Os LEDs são feitos de materiais **semicondutores.** Substituindo alguns dos seus átomos por outros em um processo chamado de **dopagem**, é possível controlar a **cor** emitida pelo dispositivo.




| Semicondutor | Cor da luz | Comprimento de onda |
|---|---|---|
| Arsenieto de gálio e alumínio | Infravermelha | 880 nm |
| Arsenieto de gálio e alumínio | Vermelha | 645 nm |
| Fosfato de alumínio, índio e gálio | Amarela | 595 nm |
| Fosfato de gálio | Verde | 565 nm |
| Nitreto de gálio | Azul | 430 nm |

# Componentes - LED

**Tabela de tensão e corrente para os LED`s**

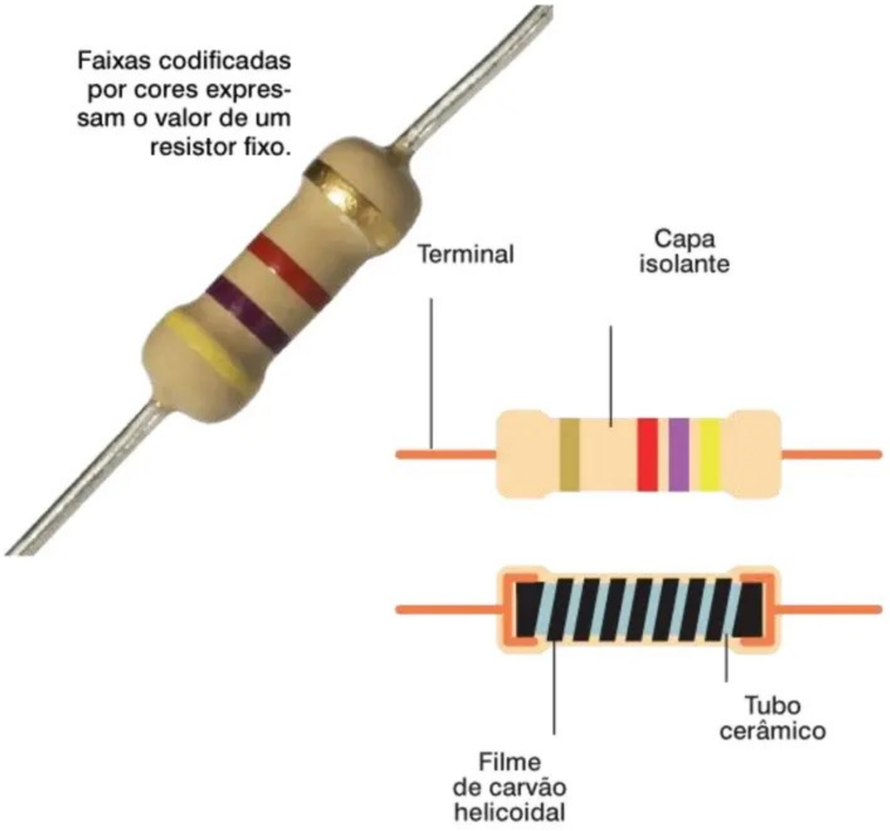
| Cor dos LEDs | Tensão mínima e máxima para o LED | Corrente máxima para o LED |
|---|---|---|
| Vermelho | 1,8V - 2,0V | 20mA |
| Amarelo | 1,8V - 2,0V | 20mA |
| Laranja | 1,8V - 2,0V | 20mA |
| Verde | 2,0V - 2,5V | 20mA |
| Azul | 2,5V - 3,0V | 20mA |
| Branco | 2,5V - 3,0V | 20mA |

# Componentes - Resistor

- O **resistor** é um componente elétrico **passivo** que tem a função primária de limitar o fluxo da corrente elétrica em um circuito.

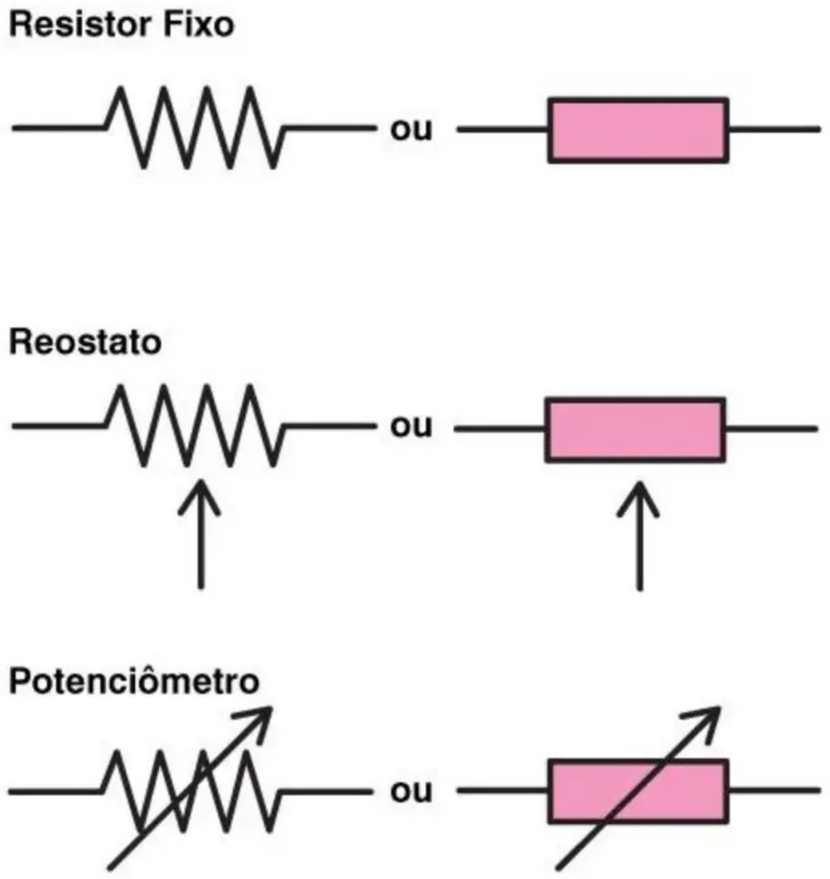
- O **tipo de material** do qual é feito o resistor também é muito importante. Os tipos variam e podem ser de **carvão** ou compostos de carbono que são construídos com uma mistura de uma **cerâmica** não condutora e partículas finas de carbono.

- Os **reostatos** são resistores de resistência variável. Lâmpadas que possuem seu brilho variável e os botões de volume dos aparelhos de som são exemplos de aplicações de reostatos.

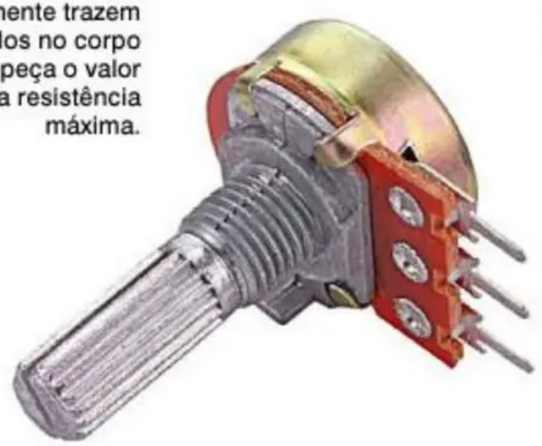


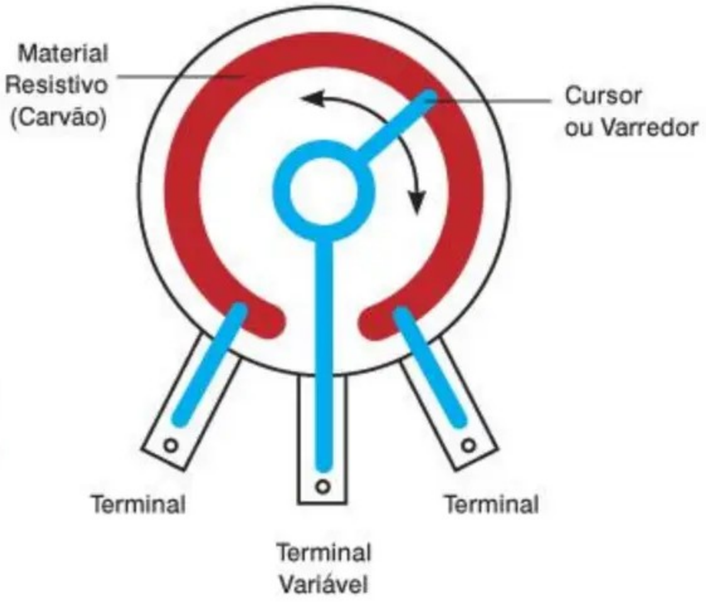
# Componentes - Resistor

- **SIMBOLOGIA**




Potenciômetros geralmente trazem gravados no corpo da peça o valor de sua resistência máxima.

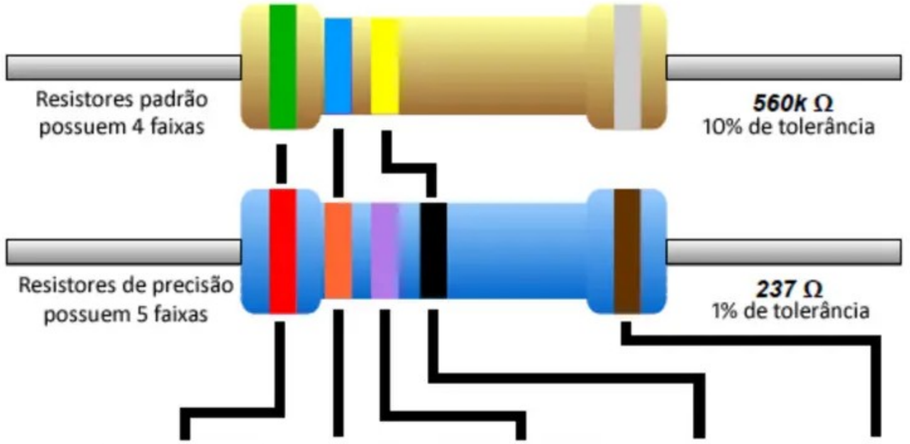




# Componentes - Resistor

## Código de Cores

A extremidade com mais faixas deve apontar para a esquerda



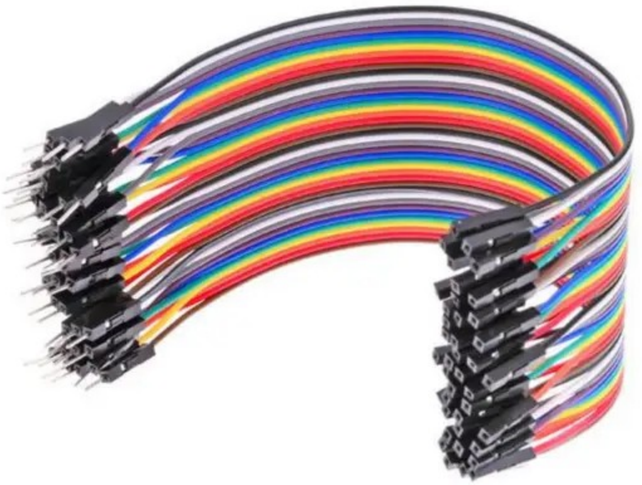

| Cor | 1ª Faixa | 2ª Faixa | 3ª Faixa | Multiplicador | Tolerância |
|---|---|---|---|---|---|
| Preto | 0 | 0 | 0 | x 1 Ω | |
| Marrom | 1 | 1 | 1 | x 10 Ω | +/- 1% |
| Vermelho | 2 | 2 | 2 | x 100 Ω | +/- 2% |
| Laranja | 3 | 3 | 3 | x 1K Ω | |
| Amarelo | 4 | 4 | 4 | x 10K Ω | |
| Verde | 5 | 5 | 5 | x 100K Ω | +/- .5% |
| Azul | 6 | 6 | 6 | x 1M Ω | +/- .25% |
| Violeta | 7 | 7 | 7 | x 10M Ω | +/- .1% |
| Cinza | 8 | 8 | 8 | | +/- .05% |
| Branco | 9 | 9 | 9 | | |
| Dourado | | | | x .1 Ω | +/- 5% |
| Prateado | | | | x .01 Ω | +/- 10% |

# Componentes - Jumpers

- Os **Jumpers** são pequenos fios condutores que podem ser conectados a uma protoboard para interligar dois pontos do circuito em projetos eletrônicos, geralmente utilizados em conexões com Arduino, Raspberry Pi, entre outros.

- Pode ser feitos com conexões do tipo macho-fêmea, macho-macho ou fêmea-fêmea.





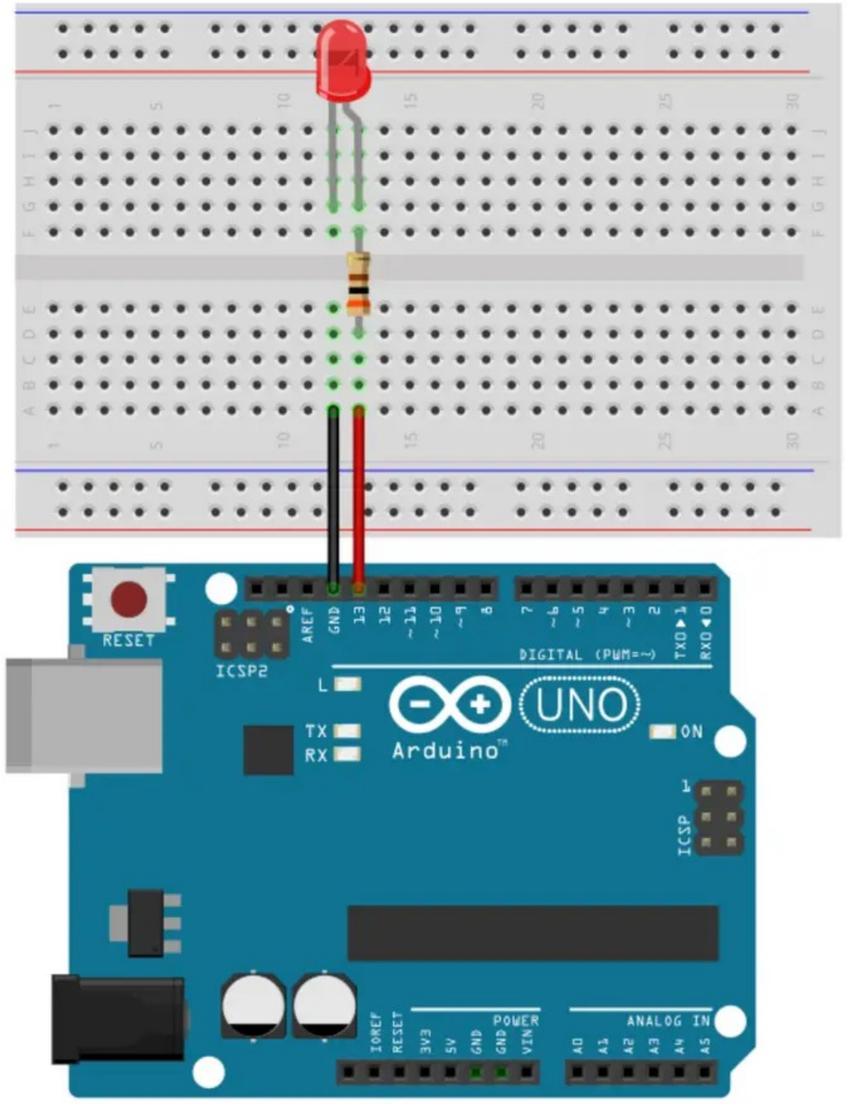
# Projeto Pisca LED

- Você precisará dos seguintes materiais para o desenvolvimento dessa atividade:

✓ 1x Placa Arduino + Cabo USB
✓ 1x LED 5mm
✓ 1x Resistor 300Ω
✓ 1x Protoboard
✓ Jumpers (fios)

# Projeto Pisca LED - Diagrama

Diagrama do projeto Pisca LED.



# Projeto Pisca LED – Código Fonte

```
/**
  Projeto PISCA LED
*/
// declaração da variável meu_led
int meu_led = 13;

void setup() {
  // use o comando pinMode() para atribuir meu_led como um pino de saída (OUTPUT)
  pinMode(meu_led, OUTPUT);
}
void loop() {
  digitalWrite(meu_led, HIGH); // faça meu_led igual a HIGH (ligue meu_led)
  delay(1000); // espere por 1s (1000ms)
  digitalWrite(meu_led, LOW); // faça meu_led igual a LOW (desligue meu_led)
  delay(1000); // espere por 1s
} // retorne para o início do loop
```

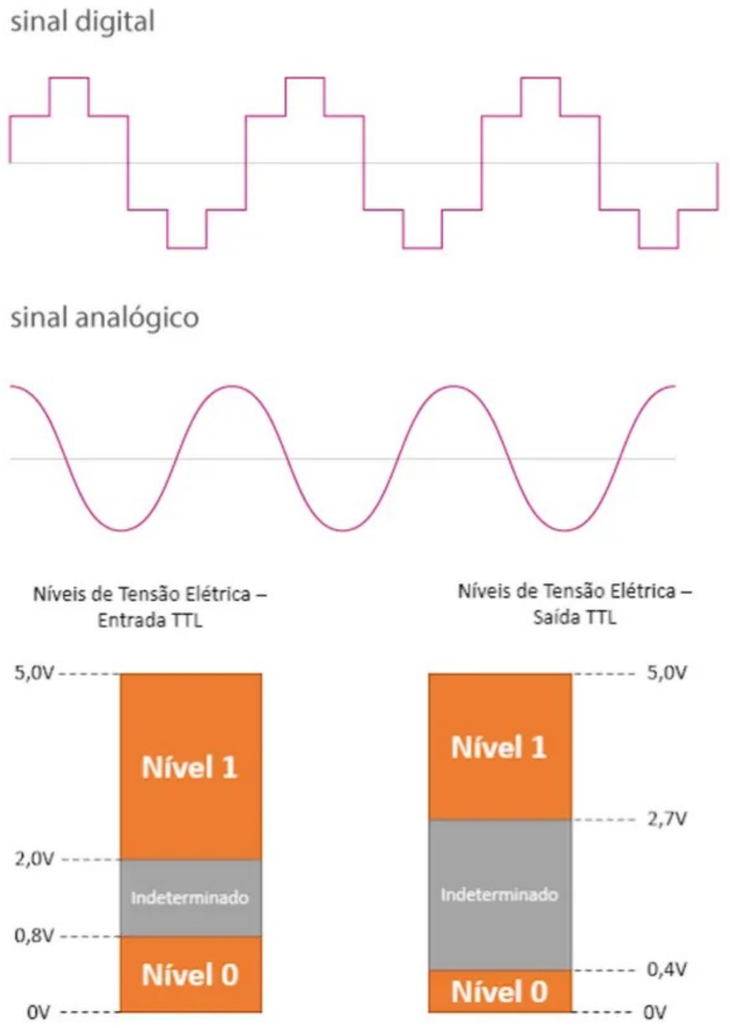
# Sinais Digitais x Sinais Analógicos

- **Sinais**

  Vários são os tipos de sinal que o Arduino consegue ler e escrever, mas eles podem ser diferenciados em dois principais grupos: **digital** e **analógico.** Um sinal digital é **+5 V ou 0 V**, mas um sinal analógico pode ser qualquer tensão linear entre **0 V e +5 V**.

- **Sinais digitais**

  No Arduino, todo **pino digital** pode ser configurado como uma **ENTRADA(INPUT)** ou **SAÍDA(OUTPUT)** por meio do comando **pinMode(pin, modo)** na função **setup().** Um sinal digital pode ter somente dois estados no Arduino: **ALTO (HIGH)** ou **BAIXO (LOW).** Isso ocorre independentemente de o sinal digital ser uma entrada ou uma saída. Quando um pino está em 5 V, é considerado alto; quando está em 0 V ou GND, é considerado baixo.



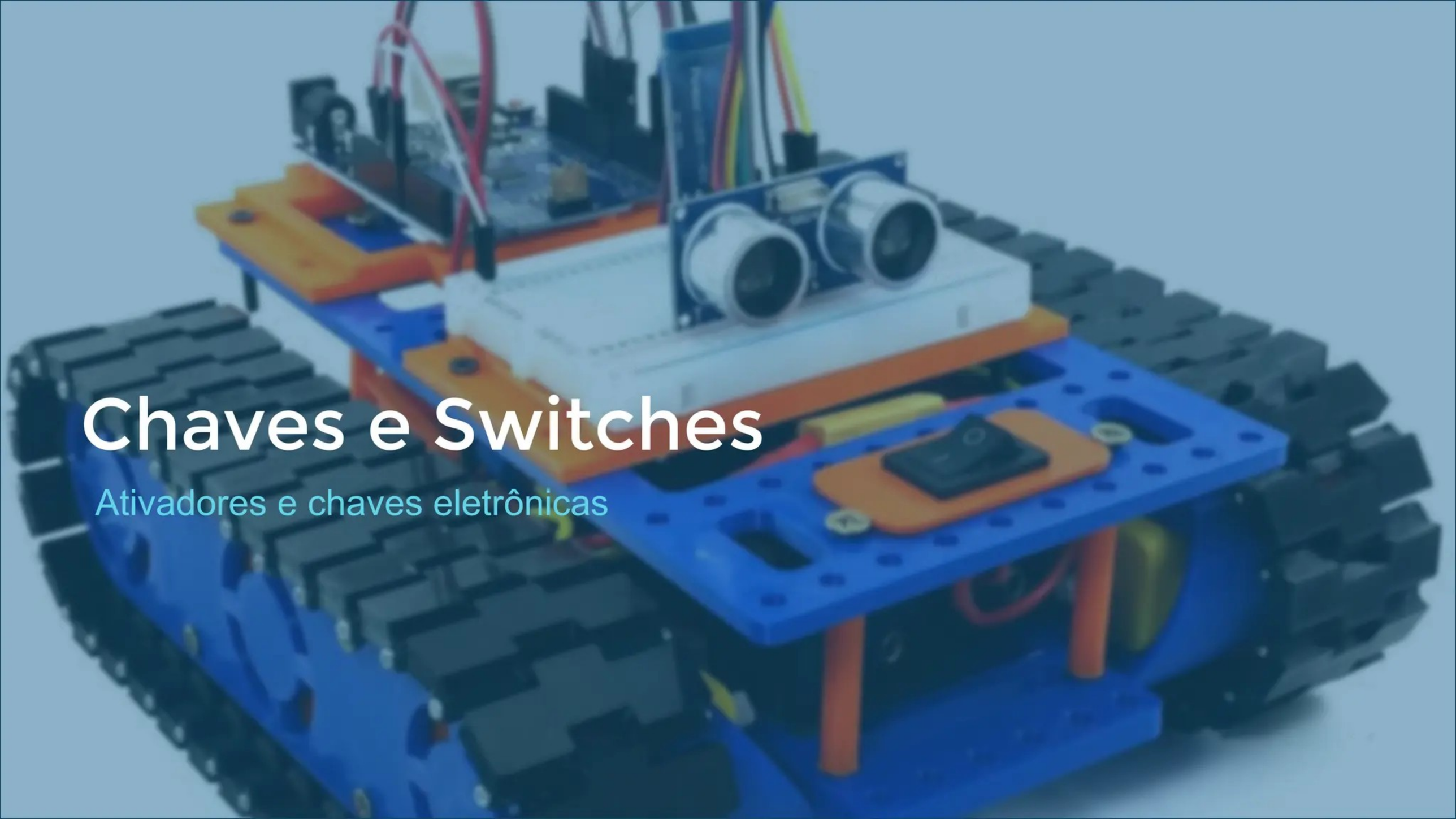
# Entradas Digitais x Saídas Digitais

- **Entradas digitais:** úteis se você quiser determinar quando um botão foi pressionado (isto é, um sensor de colisão) e se uma chave está **ligada** ou **desligada** ou se quiser ler um pulso em um sensor para determinar seu valor oculto.
- Para determinar se uma entrada é alta ou baixa, use o comando **digitalRead(pin).** Um sinal de entrada digital nem sempre pode contar com a disponibilidade total de 5 V.
- Desse modo, o limite para conduzir um pino de entrada para alto é 3 V, e qualquer valor abaixo desse limiar é considerado baixo.

# Entradas Digitais x Saídas Digitais

- **A saída digital** é igualmente simples, mas pode ser usada para fazer tarefas complexas. Se você tem um Arduino, já viu o sketch “Hello, World” (“Olá, Mundo!”), que simplesmente **pisca o LED** no pino **D13** embutido na placa – esse é o uso mais simples de uma **saída digital**. **Todo pino** do Arduino é capaz de fornecer ou de drenar uma corrente de **40 mA** em **5 V.**
- A função **digitalWrite(**pin, valor**)** aciona um **valor HIGH** ou **LOW** em um pino digital.

```
void setup() {
  pinMode(13, OUTPUT);    // configura o pino digital 13 como saída
}
void loop() {
  digitalWrite(13, HIGH); // ativa o pino digital 13
  delay(1000); // espera por um segundo
  digitalWrite(13, LOW);  // desativa o pino digital 13
  delay(1000);  // espera por um segundo
}
```



# Chaves e Switches

Ativadores e chaves eletrônicas

# Chaves e Switches

- Em muitos projetos precisamos alterar parâmetros de funcionamento durante o uso, uma **chave seletora** é uma ótima opção de interface, assim como um **push button.**

- Um exemplo clássico de é **chave HH** de alavanca com três posições é ideal para aplicações de comutar alimentação, ou controle de motores DC.

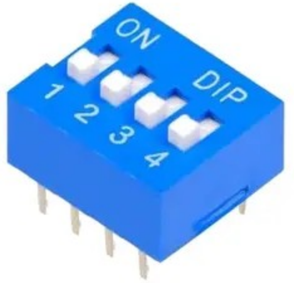
- A **Chave DIP Switch** é um componente utilizado em projetos eletrônicos, composto por chaves com 2 posições (ligado/desligado), cada uma funcionando como um interruptor.



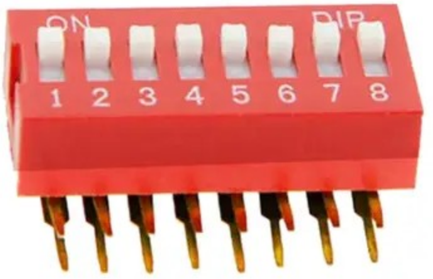
(a) Chave seletora HH



(b) Push button




(c) Chave DIP 4 vias




(d) Chave DIP 8 vias



# Resistores de Pull-down e Pull-up

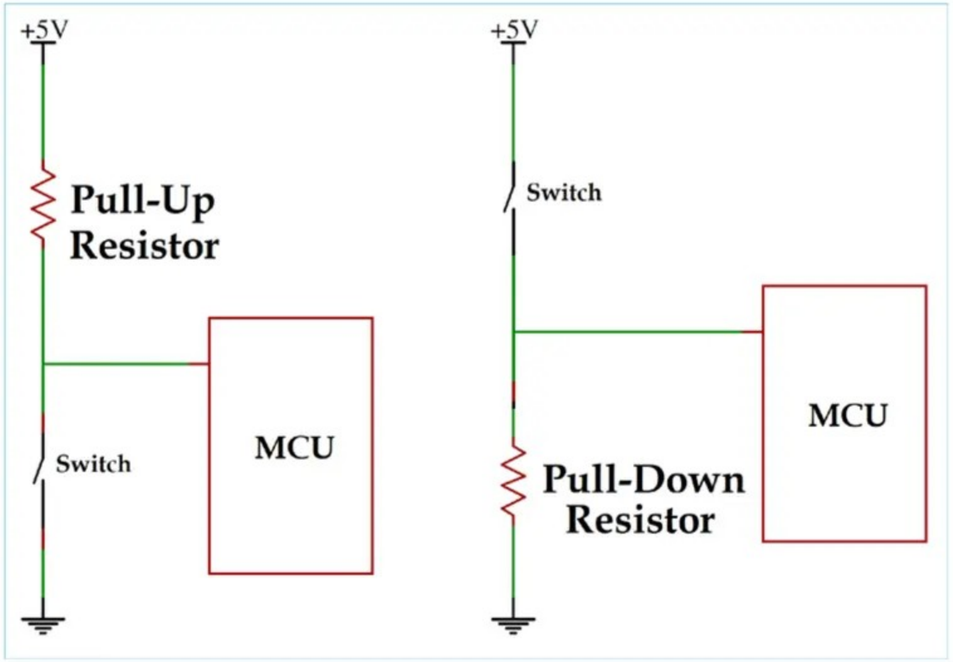
Resolvendo o problema de circuitos
com flutuação.

# Resistor Pull Down e Pull Up

- Em praticamente todo projeto, é necessário que alguma leitura digital seja feita. E é extremamente ruim ler valores errados, e isso pode ocorrer por causa do pino não estar conectado a nada em determinado momento. Portanto, é necessário usar um resistor **pull-up ou pull-down** para resolver o problema.

- O resistor pull-up ou pull-down é basicamente um resistor que fica ligado junto ao sinal que você deseja ler. Ele serve para garantir que este sinal (a tensão dele), fique em um valor conhecido enquanto o pino não recebe nenhuma tensão.

- Um conceito importante é o de **“pino flutuando”**, que é o estado que o pino fica sem receber nada. Isso ocorre, pois o pino não está ligado a nada e não há como saber qual sinal ele está recebendo, se é sinal de nível alto (2~5v) ou sinal de nível baixo (0~1v).




a) Diagrama dos resistores pull down e pull up

“A corrente elétrica sempre tende a ir para o local de menor resistência em um circuito.”



# Associação de Resistores

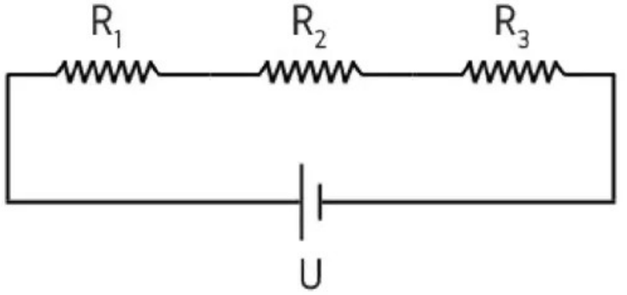
Associação paralela, série e mista de resistores

# Associação de Resistores

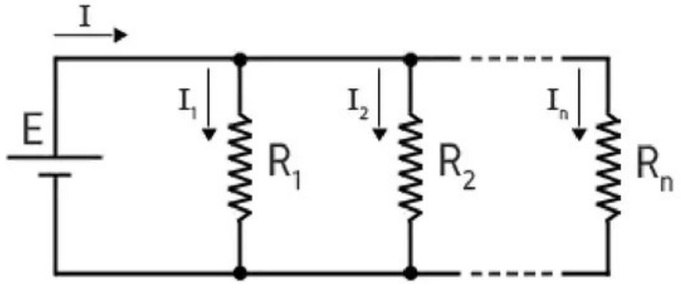
- Associação de resistores é o circuito elétrico formado por dois ou mais resistores ligados em série, paralelo ou ainda, em uma associação mista.

- Na **associação de resistores em série,** os resistores são ligados **em sequência**. Isso faz com que a corrente elétrica seja a mesma ao longo do circuito, enquanto a tensão elétrica varia.

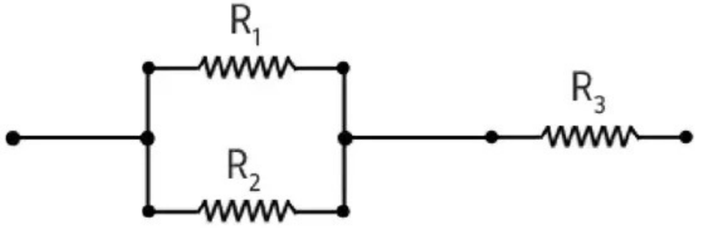
- **Na associação de resistores em paralelo**, todos os resistores estão submetidos a uma mesma tensão elétrica. Sendo a corrente elétrica dividida pelo circuito.



a) Associação em série de resistores



b) Associação em paralelo de resistores



c) Associação mista de resistores

# Associação de Resistores

- **Resistência equivalente ($R_{eq}$)** de um circuito corresponde à soma das resistências presente no circuito;
- A fórmula usada para calcular a **resistência equivalente** para resistores **em série:**

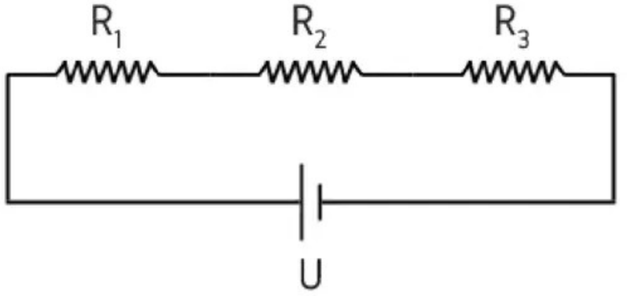
$$R_{eq} = R_1 + R_2 + R_3 + ... + R_n$$

- A fórmula usada para calcular a **resistência equivalente** para resistores **em paralelo**:

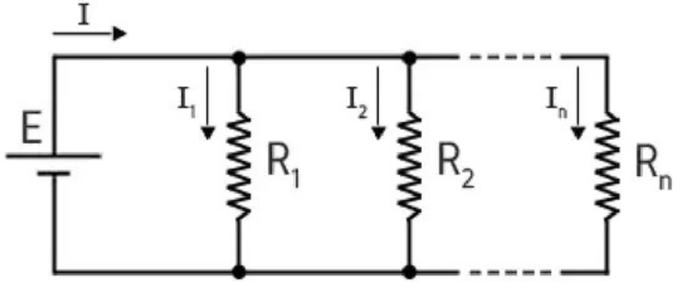
$$\frac{1}{R_{eq}} = \frac{1}{R_1} + \frac{1}{R_2} + \cdots + \frac{1}{R_i} \text{ , para } \mathbf{i} \text{ resistores;}$$

$$R_{eq} = \frac{R_1 \times R_2}{R_1 + R_2} \text{ , para } \mathbf{dois} \text{ resistores;}$$

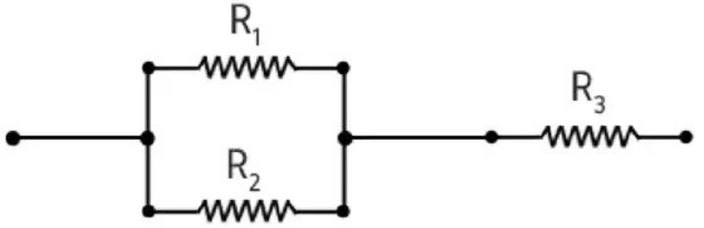
$$R_{eq} = \frac{R}{N} \text{ , para } \mathbf{N} \text{ resistores idênticos.}$$



a) Associação em série de resistores



b) Associação em paralelo de resistores



c) Associação mista de resistores

# Associação de Resistores

**(Exercício)-**Quatro resistores de resistências iguais a **10 Ω, 20 Ω, 30 Ω e 40 Ω** são ligados em **série** e depois em **paralelo**. Os valores obtidos para a **resistência equivalente** em cada um desses casos, são, respectivamente, iguais a:

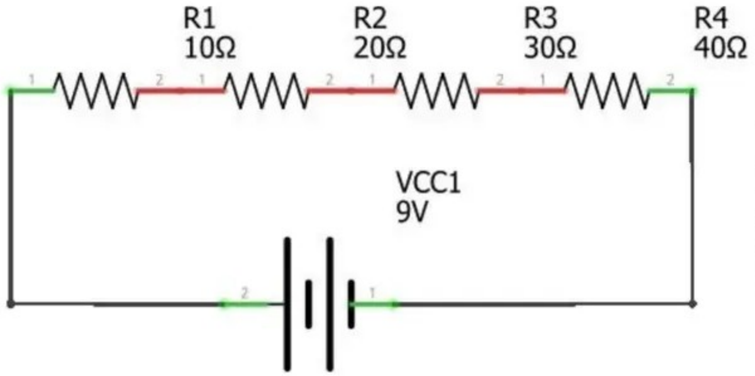
a) 150 Ω e 36 Ω

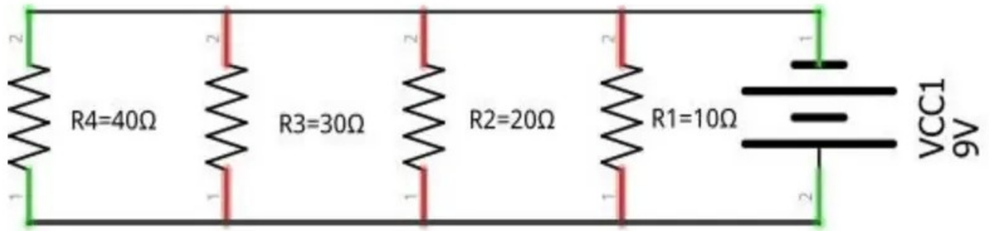
b) 10 Ω e 92 Ω

c) 100 Ω e 4,8 Ω

d) 15 Ω e 12 Ω

e) 30 Ω e 90 Ω




**(Resolução)**

$$\mathbf{R_{eq} = R_1 + R_2 + R_3 + ... + Rn}$$

$$\mathbf{R_{eq} = 10 + 20 + 30 + 40 = 100\ \Omega}$$

$$\frac{\mathbf{1}}{\boldsymbol{R_{eq}}} = \frac{\mathbf{1}}{\boldsymbol{R_1}} + \frac{\mathbf{1}}{\boldsymbol{R_2}} + \cdots + \frac{\mathbf{1}}{\boldsymbol{R_n}} = \frac{1}{10} + \frac{1}{20} + \frac{1}{30} + \frac{1}{40} \rightarrow$$

$$\frac{1}{R_{eq}} = \frac{12 + 6 + 4 + 3}{120} \rightarrow \frac{1}{R_{eq}} = \frac{25}{120} \rightarrow R_{eq} = \frac{120}{25} = \mathbf{4,8\ \Omega}$$

# Associação de Resistores

**(Exercício)-**Quatro resistores de resistências iguais a **10 kΩ** são ligados em **paralelo**. O valor obtido para a **resistência equivalente** nesse caso é igual a:

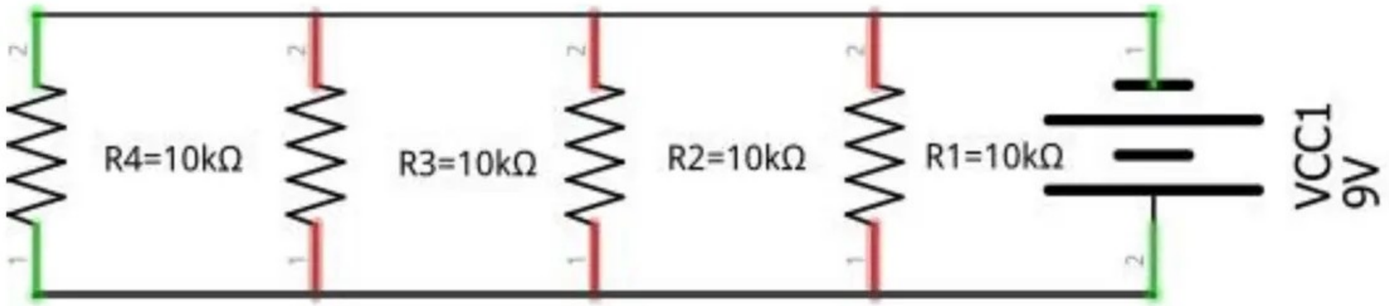
a) 6 kΩ

b) 10 kΩ

c) 4,8 kΩ

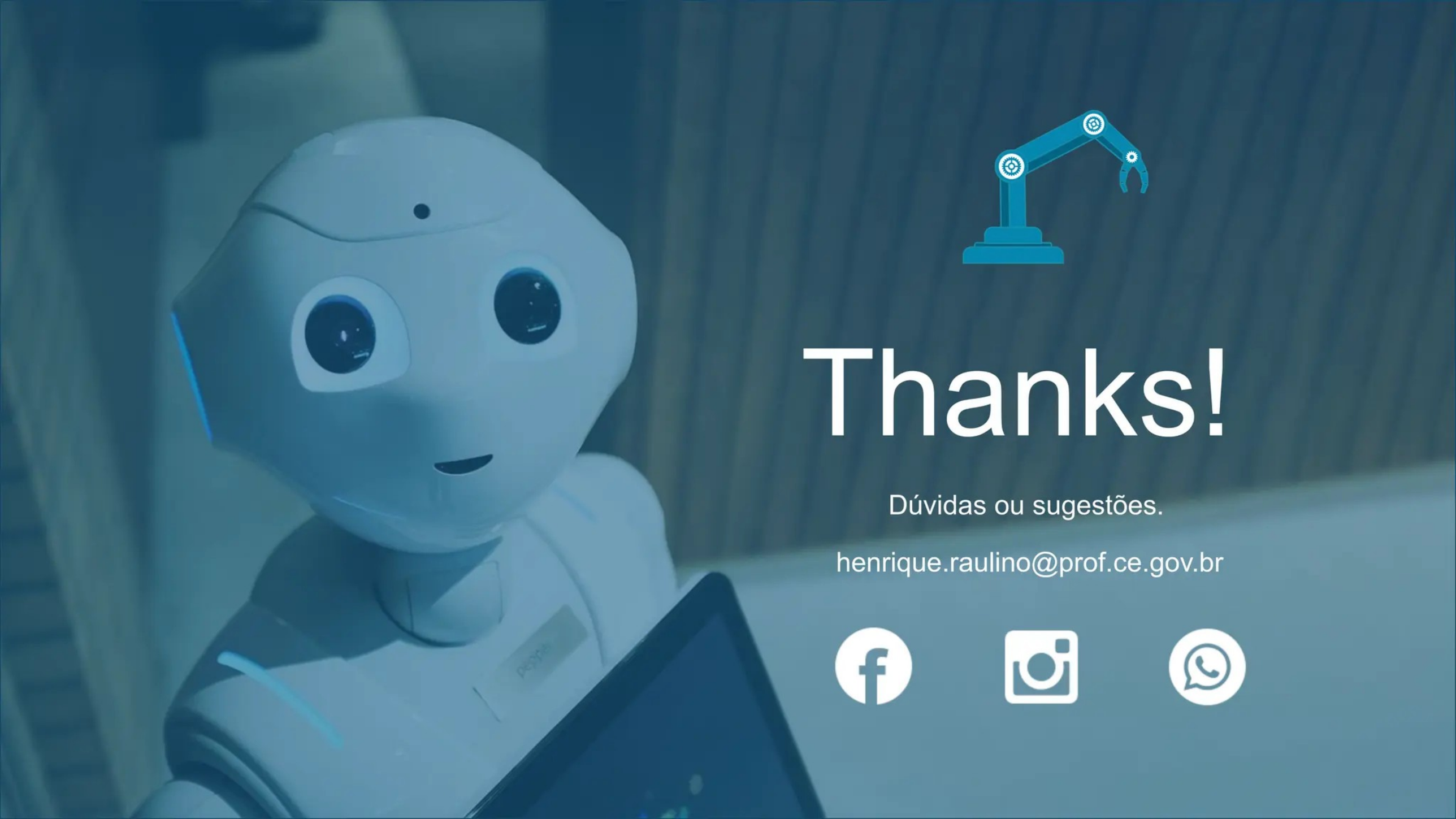
d) 2,5 kΩ

e) 20 kΩ




**(Resolução)**

$$\boldsymbol{R_{eq}} = \frac{\boldsymbol{R}}{\boldsymbol{N}} = \frac{10}{4} = \mathbf{2,5\ k\Omega}$$



# Thanks!

Dúvidas ou sugestões.

henrique.raulino@prof.ce.gov.br